

Curso

Introducción a los servicios de AWS

Módulo 1:

Fundamentos de la nube y servicios principales de AWS

Unidad 4:

Servicios de Aplicación y Almacenamiento



Presentación

En esta unidad profundizaremos en una serie de servicios clave de Amazon Web Services (AWS) orientados al almacenamiento, la distribución de contenido y el despliegue de aplicaciones. A diferencia de la unidad anterior, enfocada en cómputo, aquí analizaremos cómo AWS permite gestionar datos, acelerar la entrega de contenido y simplificar el desarrollo y despliegue de aplicaciones modernas, manteniendo altos niveles de escalabilidad, disponibilidad y eficiencia.

Comenzaremos analizando Amazon S3 (Simple Storage Service), el servicio de almacenamiento de objetos de AWS. Se explorará cómo S3 permite almacenar y recuperar grandes volúmenes de datos de manera segura, duradera y escalable. También se estudiarán conceptos clave como buckets, objetos, políticas de acceso, almacenamiento por clases (Standard, IA, Glacier) y casos de uso típicos como data lakes, backups, hosting de sitios estáticos y almacenamiento de contenido multimedia.

Luego abordaremos Amazon CloudFront, el servicio de red de distribución de contenido (CDN) de AWS. Se analizará cómo CloudFront permite acelerar la entrega de contenido web (imágenes, videos, APIs, sitios estáticos) mediante una red global de servidores (edge locations), reduciendo la latencia y mejorando la experiencia del usuario final. También se revisará su integración con S3, balanceadores de carga y aplicaciones web para optimizar el rendimiento y la seguridad.

A continuación, exploraremos Amazon Elastic Beanstalk, una plataforma como servicio (PaaS) que simplifica el despliegue y la gestión de aplicaciones. Este servicio permite a los desarrolladores subir su código sin preocuparse por la infraestructura subyacente, ya que AWS se encarga automáticamente del aprovisionamiento de recursos, balanceo de carga, escalado y monitoreo. Se analizarán sus ventajas en términos de productividad, facilidad de uso y soporte para distintos lenguajes y frameworks.

Posteriormente, introduciremos AWS Amplify, una herramienta orientada al desarrollo de aplicaciones web y móviles modernas. Amplify facilita la creación de frontends integrados con servicios backend de AWS, permitiendo implementar autenticación, almacenamiento, APIs y hosting de manera rápida. Se estudiará cómo esta herramienta acelera el desarrollo full-stack y simplifica la integración con servicios como Lambda, API Gateway o DynamoDB.

Finalmente, se presentará una introducción a la automatización del desarrollo de software, abordando conceptos clave como integración continua (CI), entrega continua (CD) y pipelines de despliegue. Se analizará cómo AWS ofrece herramientas para automatizar el ciclo de vida del software, mejorando la calidad, reduciendo errores manuales y acelerando el time-to-market de las aplicaciones.

En conjunto, esta unidad permitirá comprender cómo almacenar, distribuir y desplegar aplicaciones en AWS de forma eficiente, sentando las bases para construir soluciones modernas, escalables y orientadas a la experiencia del usuario



Objetivos

Que los participantes logren...

- Comprender el funcionamiento de Amazon S3 como servicio de almacenamiento de objetos y su rol en la gestión de datos en la nube.
- Analizar cómo Amazon CloudFront permite acelerar la entrega de contenido web mediante el uso de una red de distribución global (CDN).
- Identificar las características y casos de uso de Amazon Elastic Beanstalk como plataforma para el despliegue simplificado de aplicaciones.
- Explorar las capacidades de AWS Amplify para el desarrollo de aplicaciones web y móviles modernas, integrando frontend y servicios backend.
- Introducir los conceptos fundamentales de automatización del desarrollo de software (CI/CD) y comprender su importancia en la entrega continua de aplicaciones en la nube.



Bloques temáticos

1. Amazon S3 (Simple Storage Service)
 - 1.1 Modelo de almacenamiento: buckets y objetos
 - 1.2 Características clave de Amazon S3
 - 1.3 Clases de almacenamiento
 - 1.4 Seguridad en S3
2. ¿Qué es Amazon CloudFront?
 - 2.1 ¿Cómo funciona CloudFront?
 - 2.2 Conceptos clave
 - 2.3 Características principales y seguridad
3. Amazon Elastic Beanstalk
 - 3.1 Funcionamiento y arquitectura interna
 - 3.2 Escalabilidad, disponibilidad y operación
 - 3.3 Configuración, integración y casos de uso
4. AWS Amplify
 - 4.1 Funcionamiento y arquitectura
 - 4.2 Capacidades principales y automatización
 - 4.3 Integración, casos de uso y posicionamiento
 - 4.4 Desarrollo de aplicaciones móviles
5. Automatización del desarrollo de software (CI/CD)
 - 5.1 ¿Cómo funciona un pipeline de CI/CD?
 - 5.2 Herramientas de CI/CD en AWS
 - 5.3 Beneficios de implementar CI/CD
 - 5.4 Casos de uso y buenas prácticas
6. Conclusión
7. Glosario de conceptos clave

1. Amazon S3 (Simple Storage Service)

Amazon S3 (Simple Storage Service) es el servicio de almacenamiento de objetos de AWS, diseñado para almacenar y recuperar grandes volúmenes de información de forma segura, escalable y altamente disponible. A diferencia de los sistemas tradicionales de almacenamiento basados en archivos o bloques, S3 utiliza un modelo orientado a objetos, lo que permite gestionar datos de manera mucho más flexible y eficiente en entornos cloud.

En términos prácticos, S3 actúa como un repositorio global donde se pueden guardar desde archivos simples, como imágenes o documentos, hasta datasets masivos utilizados en análisis de datos, machine learning o backup empresarial. Su arquitectura está pensada para escalar automáticamente sin intervención del usuario, lo que significa que no es necesario preocuparse por capacidad, discos o infraestructura subyacente.



1.1. Modelo de almacenamiento: buckets y objetos

El funcionamiento de S3 se basa en dos conceptos fundamentales: buckets y objetos. Un bucket es el contenedor lógico donde se almacenan los datos, y cada uno debe tener un nombre único a nivel global dentro de AWS. Dentro de estos buckets se almacenan los objetos, que representan los archivos en sí mismos.

Cada objeto no solo contiene el archivo, sino también información adicional llamada metadatos, que describe características como el tipo de contenido, fecha de creación o permisos. Además, cada objeto se identifica mediante una clave única (key), que funciona de forma similar a una ruta dentro de un sistema de archivos, aunque internamente S3 no utiliza una estructura jerárquica tradicional.

Este modelo permite organizar la información de manera flexible, simulando carpetas mediante prefijos en las claves, lo que facilita la navegación y gestión de grandes volúmenes de datos.

1.2 Características clave de Amazon S3

Amazon S3 se destaca por una serie de características que lo convierten en un servicio central dentro de AWS. En primer lugar, ofrece una durabilidad extremadamente alta (99.999999999%), lo que significa que los datos están replicados automáticamente en múltiples dispositivos y ubicaciones dentro de una región. Esto reduce significativamente el riesgo de pérdida de información.

Otra característica importante es su alta disponibilidad, ya que los datos pueden ser accedidos en cualquier momento mediante protocolos estándar como HTTP o HTTPS. Además, S3 es completamente escalable, lo que permite almacenar

desde unos pocos megabytes hasta petabytes de información sin necesidad de realizar configuraciones adicionales.

También es un servicio altamente integrado con el resto del ecosistema AWS. Por ejemplo, puede utilizarse junto con servicios de cómputo como EC2 o Lambda, bases de datos, herramientas de análisis de datos o servicios de distribución de contenido como CloudFront.

1.3 Clases de almacenamiento

Uno de los aspectos más importantes de S3 es su modelo de costos basado en el uso y la frecuencia de acceso a los datos. Para optimizar esto, AWS ofrece diferentes clases de almacenamiento, cada una diseñada para distintos escenarios.

- **S3 Standard:** pensado para datos de acceso frecuente, con alta disponibilidad y bajo tiempo de acceso.
- **S3 Intelligent-Tiering:** mueve automáticamente los datos entre niveles según su uso, optimizando costos sin intervención manual.
- **S3 Standard-IA (Infrequent Access):** ideal para datos que se acceden ocasionalmente, con menor costo de almacenamiento pero mayor costo de acceso.
- **S3 Glacier y Glacier Deep Archive:** diseñados para almacenamiento a largo plazo y archivado, con costos muy bajos pero tiempos de recuperación más altos.

La elección de la clase adecuada permite equilibrar rendimiento y costo, siendo un factor clave en arquitecturas eficientes.

Para más información, pueden acceder a la documentación oficial del sitio de AWS: <https://aws.amazon.com/es/s3/storage-classes/>

1.4 Seguridad en S3

La seguridad en Amazon S3 es altamente configurable y permite controlar quién puede acceder a los datos y de qué manera. Esto se logra mediante distintos mecanismos que pueden combinarse según las necesidades:

- **Políticas de acceso (Bucket Policies e IAM)**
- **Control de acceso a nivel de objeto**
- **Cifrado de datos en reposo y en tránsito**
- **Bloqueo de acceso público (muy importante por defecto)**

Estos mecanismos permiten construir entornos seguros tanto para aplicaciones internas como para contenido público, como sitios web estáticos.

Amazon S3 es uno de los servicios más versátiles de AWS, lo que se refleja en la gran cantidad de escenarios donde puede utilizarse. Es común emplearlo para almacenamiento de archivos y backups, ya que ofrece una solución confiable y económica para resguardar información crítica. También es ampliamente utilizado para el hosting de sitios web estáticos, donde los archivos HTML, CSS y JavaScript se sirven directamente desde S3.

En entornos de datos, S3 es la base de muchos data lakes, donde se almacenan grandes volúmenes de información para su posterior procesamiento con herramientas de analítica. Asimismo, se utiliza para almacenar contenido multimedia, como imágenes y videos, que luego pueden distribuirse globalmente mediante servicios como CloudFront.

2. ¿Qué es Amazon CloudFront?

Amazon CloudFront es el servicio de red de distribución de contenido (CDN, Content Delivery Network) de AWS, diseñado para entregar contenido a usuarios finales con baja latencia, alta velocidad y alta disponibilidad.

Su principal objetivo es mejorar la experiencia del usuario reduciendo el tiempo que tarda en cargarse un sitio web, una API o cualquier tipo de contenido distribuido a través de internet.

A diferencia de un servidor tradicional que responde todas las solicitudes desde una única ubicación, CloudFront utiliza una red global de servidores distribuidos geográficamente, conocidos como **edge locations** que almacenan copias del contenido más cerca de los usuarios. Esto permite que las solicitudes se resuelvan desde el punto más cercano, reduciendo la distancia física y, por lo tanto, el tiempo de respuesta.



2.1. ¿Cómo funciona CloudFront?

El funcionamiento de CloudFront se basa en el concepto de **caché distribuida**. Cuando un usuario solicita un recurso (por ejemplo, una imagen o un archivo HTML), la solicitud es redirigida automáticamente a la ubicación de borde más cercana.

Si el contenido ya se encuentra en caché en esa ubicación, CloudFront lo entrega directamente, logrando una respuesta casi inmediata. En caso de que no esté disponible, CloudFront lo obtiene desde el origen (por ejemplo, un bucket de S3, una instancia EC2 o un load balancer), lo almacena temporalmente en caché y luego lo entrega al usuario.

Este mecanismo permite que, a medida que más usuarios acceden al contenido, el sistema se vuelva más eficiente, ya que reduce la cantidad de solicitudes que llegan al origen.

2.2 Conceptos clave

Para comprender mejor cómo se configura y utiliza este servicio, es importante entender algunos de sus componentes principales:

- **Distribución (Distribution):** es la configuración principal de CloudFront, donde se define qué contenido se distribuye, desde dónde (origen) y bajo qué reglas.
- **Origen (Origin):** es la fuente del contenido. Puede ser un bucket de S3, una instancia EC2, un Elastic Load Balancer o incluso un servidor externo.
- **Edge Locations:** son los puntos de presencia distribuidos globalmente donde se almacena el contenido en caché.

- **Cache Behavior:** define cómo CloudFront maneja las solicitudes, incluyendo reglas de caché, protocolos y headers.

2.3 Características principales y seguridad

CloudFront ofrece una serie de capacidades que lo convierten en un componente clave para aplicaciones modernas:

- **Red global de distribución**, con cientos de ubicaciones en todo el mundo
- **Baja latencia**, al servir contenido desde la ubicación más cercana al usuario
- **Escalabilidad automática**, capaz de manejar grandes volúmenes de tráfico
- **Compatibilidad con contenido estático y dinámico**, incluyendo APIs

Además, se integra de forma nativa con otros servicios de AWS, lo que facilita su adopción dentro de arquitecturas existentes.

Por otro lado, uno de los aspectos más importantes de CloudFront es su enfoque en la seguridad. El servicio incorpora múltiples capas de protección que permiten resguardar tanto el contenido como la infraestructura:

- **Soporte para HTTPS**, asegurando la transmisión cifrada de datos
- **Integración con AWS Shield**, que protege contra ataques DDoS
- **AWS WAF (Web Application Firewall)**, que permite filtrar tráfico malicioso
- **Control de acceso mediante firmas y URLs firmadas**, para restringir contenido privado

CloudFront es ampliamente utilizado en escenarios donde el rendimiento y la experiencia del usuario son críticos. Uno de los casos más comunes es la distribución de sitios web estáticos, donde archivos como HTML, CSS, JavaScript e imágenes se sirven rápidamente a usuarios en todo el mundo.

También es muy utilizado para el **streaming de contenido multimedia**, como video bajo demanda o transmisiones en vivo, donde la baja latencia es fundamental. En el caso de APIs, CloudFront puede actuar como una capa de aceleración, reduciendo tiempos de respuesta y descargando trabajo del backend.

Otro uso frecuente es la combinación con Amazon S3 para crear arquitecturas altamente escalables y eficientes, donde S3 almacena el contenido y CloudFront lo distribuye globalmente.

3. Amazon Elastic Beanstalk

Amazon Elastic Beanstalk es un servicio de tipo Platform as a Service (PaaS) que permite desplegar y administrar aplicaciones web sin necesidad de gestionar directamente la infraestructura. Su objetivo principal es simplificar el proceso de llevar una aplicación a producción, reduciendo la carga operativa asociada a servidores, redes, escalabilidad y monitoreo.

A diferencia de trabajar directamente con servicios como EC2, donde el usuario debe configurar manualmente cada componente, Elastic Beanstalk automatiza todo el proceso de aprovisionamiento. Esto permite que los desarrolladores se enfoquen en el desarrollo del código y no en la gestión de la infraestructura.

Sin embargo, a diferencia de otros enfoques más abstractos como el serverless, Elastic Beanstalk no oculta completamente los recursos subyacentes. Internamente utiliza servicios como EC2, Auto Scaling y Elastic Load Balancing, lo que permite mantener cierto nivel de control y visibilidad sobre la arquitectura desplegada.

Este posicionamiento lo convierte en una solución intermedia ideal: combina facilidad de uso con flexibilidad, siendo especialmente útil para equipos que quieren acelerar el desarrollo sin perder completamente el control técnico.

3.1 Funcionamiento y arquitectura interna

El funcionamiento de Elastic Beanstalk se basa en un flujo simple: el usuario sube su aplicación, selecciona una plataforma y el servicio se encarga del resto. A partir de ese momento, AWS crea automáticamente todos los recursos necesarios para ejecutar la aplicación en un entorno productivo.

Cuando se despliega una aplicación, Elastic Beanstalk aprovisiona una arquitectura completa que suele incluir instancias EC2 donde corre la aplicación, un balanceador de carga para distribuir el tráfico, un grupo de Auto Scaling para ajustar la capacidad según la demanda, almacenamiento (como volúmenes EBS) y herramientas de monitoreo.

Desde el punto de vista organizativo, el servicio estructura los recursos en distintos niveles:

- **Application:** representa el proyecto lógico.
- **Application Version:** define versiones específicas del código.
- **Environment:** es el entorno donde se ejecuta la aplicación (por ejemplo, desarrollo o producción).

Cada entorno es completamente independiente y puede tener su propia configuración, lo que permite trabajar con múltiples versiones y entornos en paralelo. Esta estructura es clave para implementar buenas prácticas como despliegues progresivos o separación entre ambientes.

3.2 Escalabilidad, disponibilidad y operación

Uno de los pilares de Elastic Beanstalk es su capacidad de escalar automáticamente y garantizar alta disponibilidad sin intervención manual. Esto se logra gracias a su integración con servicios fundamentales de AWS.

Por un lado, utiliza **Auto Scaling Groups** para ajustar dinámicamente la cantidad de instancias en función de métricas como uso de CPU o tráfico. Esto permite que la aplicación responda correctamente tanto en momentos de baja demanda como ante picos de uso.

Por otro lado, incorpora **Elastic Load Balancing**, que distribuye el tráfico entre múltiples instancias, evitando que una sola se sobrecargue. En caso de que alguna instancia falle, el sistema la reemplaza automáticamente, manteniendo la continuidad del servicio.

Además, Elastic Beanstalk permite desplegar aplicaciones en múltiples zonas de disponibilidad dentro de una región, lo que incrementa la resiliencia ante fallos de infraestructura. También se integra con herramientas de monitoreo como CloudWatch, lo que facilita la observabilidad del sistema y la detección temprana de problemas.

Desde el punto de vista operativo, esto reduce significativamente la necesidad de intervención manual, permitiendo que la infraestructura se adapte automáticamente a las condiciones de uso.

3.3 Configuración, integración y casos de uso

A pesar de su enfoque simplificado, Elastic Beanstalk ofrece un nivel considerable de personalización. Los usuarios pueden definir configuraciones específicas como el tipo y tamaño de las instancias, reglas de escalado, variables de entorno o configuraciones de red dentro de una VPC.

También es posible extender su funcionalidad mediante archivos de configuración avanzados, lo que permite automatizar tareas adicionales o modificar el comportamiento del entorno. Esta flexibilidad es clave para adaptarse a distintos requerimientos sin perder la simplicidad inicial.

En cuanto a integración, Elastic Beanstalk se conecta fácilmente con otros servicios de AWS. Por ejemplo, puede utilizar bases de datos como RDS, almacenar archivos en S3, enviar logs a CloudWatch o integrarse con pipelines de CI/CD para automatizar despliegues. Esto lo convierte en una pieza importante dentro de arquitecturas más amplias.

En términos de uso, Elastic Beanstalk es especialmente adecuado para aplicaciones web tradicionales, APIs y proyectos que requieren un despliegue rápido. Es muy utilizado en etapas de desarrollo inicial, prototipos o MVPs, donde la velocidad de implementación es prioritaria. También es válido para aplicaciones empresariales que necesitan escalabilidad y disponibilidad, pero sin la complejidad de gestionar infraestructura desde cero.

En síntesis, permite simplificar el despliegue y la operación de aplicaciones en la nube, automatizando tareas clave sin eliminar completamente el control sobre la infraestructura. Su equilibrio entre facilidad de uso y capacidad de personalización lo convierte en una herramienta muy valiosa para equipos que buscan eficiencia, escalabilidad y rapidez en sus procesos de desarrollo y despliegue.

4. AWS Amplify

AWS Amplify es un conjunto de herramientas y servicios diseñados para facilitar el desarrollo de aplicaciones web y móviles modernas, especialmente desde el lado del frontend. Su principal objetivo es simplificar la creación de aplicaciones full-stack, permitiendo integrar rápidamente interfaces de usuario con servicios backend sin necesidad de gestionar infraestructura compleja.

A diferencia de otros servicios como Elastic Beanstalk, que se enfocan en el despliegue de aplicaciones completas, **Amplify está pensado para acelerar el desarrollo desde el lado del cliente** (frontend), integrando funcionalidades como autenticación, almacenamiento, APIs y hosting de manera sencilla y automatizada.

Este enfoque lo hace especialmente relevante en el contexto actual, donde predominan las aplicaciones tipo SPA (Single Page Applications) y aplicaciones móviles que consumen APIs. Amplify permite a los desarrolladores construir este tipo de soluciones sin necesidad de diseñar toda la arquitectura backend desde cero, reduciendo significativamente la complejidad técnica



4.1. Funcionamiento y arquitectura

El funcionamiento de AWS Amplify se basa en la integración entre el desarrollo frontend y los servicios backend de AWS. A través de herramientas específicas, los desarrolladores pueden definir funcionalidades backend (como autenticación o APIs) y conectarlas directamente con la interfaz de usuario.

Amplify ofrece varias formas de trabajo. Por un lado, dispone de una **biblioteca de cliente (Amplify Libraries)** que permite integrar servicios de AWS directamente en aplicaciones desarrolladas en frameworks como React, Angular, Vue o incluso aplicaciones móviles con React Native o Flutter. Por otro lado, ofrece herramientas de línea de comandos (CLI) que permiten crear y configurar recursos backend de manera declarativa.

A nivel de arquitectura, Amplify actúa como una capa de abstracción sobre múltiples servicios de AWS. Por ejemplo, cuando se implementa autenticación, internamente se utiliza Amazon Cognito; para APIs se puede usar API Gateway y

Lambda; para almacenamiento, S3 o DynamoDB. Todo esto ocurre de forma transparente para el desarrollador.

Además, Amplify incluye un servicio de **hosting integrado**, que permite desplegar aplicaciones frontend directamente desde repositorios como GitHub o GitLab, configurando automáticamente pipelines de build y deploy.

4.2. Capacidades principales y automatización

Uno de los aspectos más destacados de AWS Amplify es su capacidad para acelerar el desarrollo mediante la automatización de tareas comunes. El servicio permite incorporar funcionalidades complejas con muy poco esfuerzo, lo que reduce considerablemente el tiempo de desarrollo.

Entre sus capacidades más relevantes se destacan:

- **Autenticación de usuarios**, incluyendo registro, login y gestión de identidades
- **APIs (REST y GraphQL)** para comunicación entre frontend y backend
- **Almacenamiento de archivos**, integrando servicios como S3
- **Bases de datos**, generalmente mediante DynamoDB
- **Hosting y despliegue automático**, con integración a repositorios

Amplify también facilita la implementación de prácticas modernas de desarrollo, como la integración continua y el despliegue continuo (CI/CD). Cada vez que se realiza un cambio en el repositorio, el sistema puede construir y desplegar automáticamente la aplicación, lo que agiliza el ciclo de desarrollo.

Otra capacidad importante es la posibilidad de gestionar entornos (por ejemplo, desarrollo, staging y producción), lo que permite trabajar de manera organizada en equipos y evitar conflictos entre versiones.

4.3. Integración, casos de uso y posicionamiento

AWS Amplify se integra de manera natural con el ecosistema de AWS, lo que permite construir aplicaciones completas utilizando múltiples servicios sin necesidad de configurarlos manualmente. Esta integración facilita la adopción de arquitecturas modernas basadas en microservicios y eventos.

En términos de casos de uso, Amplify es ideal para aplicaciones frontend modernas, como dashboards interactivos, aplicaciones SPA, aplicaciones móviles y plataformas que requieren autenticación de usuarios y consumo de APIs. También es muy utilizado en proyectos donde se busca lanzar rápidamente un producto, como startups o MVPs.

Desde una perspectiva de posicionamiento, Amplify se ubica como una solución orientada a desarrolladores frontend que necesitan capacidades backend sin profundizar en la complejidad de la infraestructura cloud. A diferencia de Elastic Beanstalk, que está más enfocado en aplicaciones backend tradicionales, Amplify está pensado para arquitecturas modernas y desacopladas.

4.4. Desarrollo de aplicaciones móviles

AWS Amplify también está especialmente diseñado para facilitar el desarrollo de **aplicaciones móviles**, permitiendo a los desarrolladores crear apps completas

sin necesidad de gestionar directamente la infraestructura backend. Este enfoque es clave en el desarrollo moderno, donde las aplicaciones móviles requieren integrarse con servicios como autenticación, almacenamiento, notificaciones y APIs.

A través de sus librerías y herramientas, Amplify permite conectar aplicaciones móviles desarrolladas en frameworks como **React Native, Flutter, iOS (Swift) o Android (Kotlin)** con servicios backend de AWS de manera simple y estructurada. Esto significa que funcionalidades complejas, como el registro de usuarios o la sincronización de datos, pueden implementarse con pocas líneas de código.

Desde el punto de vista arquitectónico, Amplify sigue el mismo enfoque que en aplicaciones web: actúa como una capa que abstrae servicios como Cognito para autenticación, AppSync o API Gateway para APIs, y S3 o DynamoDB para almacenamiento. Esto permite construir aplicaciones móviles escalables y seguras sin necesidad de diseñar toda la lógica backend desde cero.

Además, Amplify facilita el manejo de **datos en tiempo real y sincronización offline**, lo cual es fundamental en aplicaciones móviles que pueden operar con conectividad limitada. Esto permite que los usuarios interactúen con la aplicación incluso sin conexión, sincronizando los cambios cuando se restablece el acceso a internet.

Otro aspecto relevante es la integración con pipelines de CI/CD, lo que permite automatizar el proceso de build, test y despliegue de aplicaciones móviles. Esto es especialmente útil en equipos de desarrollo donde se requiere iterar rápidamente y mantener consistencia entre versiones.

En cuanto a casos de uso, Amplify es ideal para aplicaciones móviles que requieren autenticación de usuarios, consumo de APIs, almacenamiento de archivos (como imágenes o documentos) y sincronización de datos. Es

ampliamente utilizado en aplicaciones de e-commerce, redes sociales, apps corporativas y soluciones móviles para startups.

En síntesis, AWS Amplify permite simplificar el desarrollo de aplicaciones modernas al integrar frontend y backend en una misma experiencia de desarrollo. Su capacidad para automatizar tareas, acelerar despliegues y facilitar la integración con servicios de AWS lo convierte en una herramienta clave para construir aplicaciones escalables, eficientes y alineadas con las tendencias actuales del desarrollo web y móvil.

5. Automatización del desarrollo de software (CI/CD)

La automatización del desarrollo de software mediante **CI/CD (Continuous Integration / Continuous Delivery)** es una práctica fundamental en el desarrollo moderno que busca mejorar la calidad del software y acelerar su entrega. En lugar de realizar procesos manuales y propensos a errores, CI/CD propone automatizar las distintas etapas del ciclo de vida de una aplicación.

La integración continua (**CI**) consiste en integrar de forma frecuente los cambios de código en un repositorio central. Cada vez que un desarrollador realiza un cambio, el sistema ejecuta automáticamente procesos como compilación y pruebas, lo que permite detectar errores de manera temprana.

Por otro lado, la entrega continua (**CD**) se enfoca en automatizar el proceso de despliegue, asegurando que el software esté siempre listo para ser publicado en producción. En algunos casos, esto puede extenderse a deployment continuo, donde los cambios se despliegan automáticamente sin intervención humana.

Este enfoque permite pasar de ciclos de desarrollo largos y riesgosos a procesos más ágiles, seguros y controlados.

5.1. ¿Cómo funciona un pipeline de CI/CD?

El corazón de CI/CD es el **pipeline**, que representa una **serie de pasos automatizados** que el código recorre desde que se desarrolla hasta que se despliega.

Un pipeline típico incluye varias etapas:

- **Build (compilación):** se prepara el código para su ejecución, generando artefactos como aplicaciones o contenedores.
- **Test (pruebas):** se ejecutan pruebas automáticas para validar que el código funciona correctamente.
- **Deploy (despliegue):** se publica la aplicación en un entorno (testing, staging o producción).

Cada vez que un desarrollador realiza un cambio en el repositorio, el pipeline se ejecuta automáticamente, asegurando que el software pase por todos estos controles antes de llegar a los usuarios.

Este proceso no solo mejora la calidad del software, sino que también reduce el tiempo necesario para implementar nuevas funcionalidades o correcciones.

5.2. Herramientas de CI/CD en AWS

AWS ofrece un conjunto de servicios que permiten implementar pipelines de CI/CD de forma integrada dentro de su ecosistema. Estos servicios trabajan en conjunto para automatizar todo el flujo de desarrollo:

- **AWS CodeCommit:** repositorio de código similar a GitHub, donde se almacenan los proyectos.
- **AWS CodeBuild:** servicio que compila el código y ejecuta pruebas automáticamente.
- **AWS CodeDeploy:** herramienta que automatiza el despliegue de aplicaciones en servicios como EC2 o Lambda.
- **AWS CodePipeline:** servicio que orquesta todo el proceso, conectando las distintas etapas del pipeline.

Estos servicios permiten construir pipelines completos sin necesidad de herramientas externas, aunque también pueden integrarse con plataformas populares como GitHub, GitLab o Bitbucket.

5.3. Beneficios de implementar CI/CD

La adopción de CI/CD trae múltiples beneficios tanto a nivel técnico como organizacional. En primer lugar, permite detectar errores de manera temprana, lo que reduce el costo y el impacto de los fallos en producción.

Además, mejora significativamente la velocidad de desarrollo, ya que los despliegues pueden realizarse de forma frecuente y segura. Esto permite

responder rápidamente a cambios en el negocio o a necesidades de los usuarios.

Otro beneficio importante es la reducción de errores humanos, ya que los procesos manuales se reemplazan por automatizaciones controladas. Esto aumenta la confiabilidad del sistema y permite mantener una mayor consistencia entre entornos.

Por último, CI/CD fomenta buenas prácticas de desarrollo, como la colaboración entre equipos, el uso de pruebas automatizadas y la entrega continua de valor.

5.4. Casos de uso y buenas prácticas

CI/CD puede aplicarse en una amplia variedad de escenarios, desde pequeñas aplicaciones hasta sistemas empresariales complejos. Es especialmente útil en equipos que trabajan de forma colaborativa y necesitan integrar cambios de manera constante.

En aplicaciones web, por ejemplo, CI/CD permite desplegar nuevas versiones varias veces al día sin afectar a los usuarios. En entornos de microservicios, facilita la gestión de múltiples componentes independientes. También es clave en arquitecturas serverless, donde los cambios pueden desplegarse rápidamente en funciones individuales.

Entre las buenas prácticas más importantes se destacan:

- Mantener pipelines simples y progresivos
- Automatizar pruebas desde etapas tempranas
- Utilizar entornos separados (desarrollo, testing, producción)
- Monitorear los despliegues y resultados

En síntesis, CI/CD representa un cambio fundamental en la forma de desarrollar y desplegar software. Al automatizar procesos clave, permite mejorar la calidad, reducir errores y acelerar la entrega de aplicaciones.

En el contexto de AWS, estas prácticas se integran de manera natural con los servicios de la plataforma, facilitando la construcción de soluciones modernas, escalables y eficientes. Comprender estos conceptos es clave para cualquier profesional que busque trabajar con tecnologías cloud y desarrollo de software en entornos actuales.

5. Conclusión

En esta unidad exploramos un conjunto de servicios fundamentales de AWS orientados al almacenamiento, la distribución de contenido y el desarrollo de aplicaciones modernas. A lo largo del recorrido, comprendimos cómo **Amazon S3** permite almacenar grandes volúmenes de datos de forma segura, escalable y con alta durabilidad, convirtiéndose en la base de múltiples arquitecturas en la nube.

También analizamos el rol de **Amazon CloudFront** como red de distribución de contenido, entendiendo cómo mejora el rendimiento y la experiencia del usuario al reducir la latencia mediante el uso de ubicaciones distribuidas globalmente. Este servicio resulta clave en aplicaciones con usuarios geográficamente dispersos.

Por otro lado, estudiamos **Amazon Elastic Beanstalk**, que simplifica el despliegue de aplicaciones al abstraer la infraestructura, y **AWS Amplify**, que facilita el desarrollo de aplicaciones web y móviles modernas mediante la integración rápida de frontend y backend. Ambos servicios reflejan cómo AWS permite adaptarse a distintos niveles de complejidad y necesidades de desarrollo.

Finalmente, introducimos los conceptos de **CI/CD**, fundamentales para automatizar el ciclo de vida del software, mejorar la calidad del código y acelerar los tiempos de entrega. Estas prácticas representan un pilar clave en entornos de desarrollo modernos y se integran de forma natural con los servicios de AWS.

En conjunto, esta unidad brinda una visión integral de cómo almacenar datos, distribuir contenido y desarrollar aplicaciones en la nube, permitiendo diseñar soluciones más eficientes, escalables y orientadas a la experiencia del usuario.

6. Glosario de conceptos clave

- **Amazon S3 (Simple Storage Service):** Servicio de almacenamiento de objetos que permite guardar y recuperar datos de forma escalable, segura y duradera.
- **Bucket:** Contenedor lógico dentro de S3 donde se almacenan los objetos.
- **Objeto (Object):** Archivo almacenado en S3 junto con sus metadatos.
- **Clave (Key):** Identificador único de un objeto dentro de un bucket.
- **Clases de almacenamiento:** Niveles de almacenamiento en S3 que optimizan costos según la frecuencia de acceso (Standard, IA, Glacier, etc.).
- **Amazon CloudFront:** Servicio de CDN que distribuye contenido a usuarios finales con baja latencia mediante una red global de servidores.
- **CDN (Content Delivery Network):** Red de servidores distribuidos que almacenan contenido en caché para mejorar la velocidad de entrega.
- **Edge Location:** Ubicación física donde CloudFront almacena contenido en caché cerca del usuario final.
- **Origen (Origin):** Fuente original del contenido (por ejemplo, S3 o EC2).
- **Amazon Elastic Beanstalk:** Servicio PaaS que permite desplegar y gestionar aplicaciones sin administrar la infraestructura subyacente.
- **Environment (Entorno):** Instancia de ejecución de una aplicación en Beanstalk (desarrollo, testing o producción).

- **Auto Scaling:** Mecanismo que ajusta automáticamente la cantidad de recursos según la demanda.
- **Load Balancer:** Servicio que distribuye el tráfico entre múltiples instancias para mejorar disponibilidad y rendimiento.
- **AWS Amplify:** Herramienta que facilita el desarrollo de aplicaciones web y móviles modernas integrando frontend y backend.
- **Frontend:** Parte de la aplicación con la que interactúa el usuario.
- **Backend:** Lógica y servicios que procesan datos y gestionan la aplicación.
- **Hosting:** Servicio que permite publicar aplicaciones web en internet.
- **CI/CD (Continuous Integration / Continuous Delivery)**
Conjunto de prácticas que automatizan la integración, prueba y despliegue de software.
- **Pipeline:** Secuencia de pasos automatizados que sigue el código desde su desarrollo hasta su despliegue.
- **Build:** Proceso de compilación o preparación del código.
- **Test:** Ejecución de pruebas automáticas para validar el funcionamiento del software.
- **Deploy (Despliegue):** Publicación de la aplicación en un entorno.



Bibliografía utilizada y sugerida

- Amazon Web Services. (2025). AWS Documentation. Recuperado de <https://docs.aws.amazon.com/>
- Amazon Web Services. (2025). Amazon S3 User Guide. AWS Documentation. Recuperado de <https://docs.aws.amazon.com/s3/>
- Amazon Web Services. (2025). Amazon CloudFront Developer Guide. AWS Documentation. Recuperado de <https://docs.aws.amazon.com/cloudfront/>
- Amazon Web Services. (2025). AWS Elastic Beanstalk Developer Guide. AWS Documentation. Recuperado de <https://docs.aws.amazon.com/elasticbeanstalk/>
- Amazon Web Services. (2025). AWS Amplify Documentation. Recuperado de <https://docs.amplify.aws/>
- Amazon Web Services. (2025). AWS CodePipeline User Guide. AWS Documentation. Recuperado de <https://docs.aws.amazon.com/codepipeline/>
- Amazon Web Services. (2025). AWS CodeBuild User Guide. AWS Documentation. Recuperado de <https://docs.aws.amazon.com/codebuild/>
- Amazon Web Services. (2025). AWS CodeDeploy User Guide. AWS Documentation. Recuperado de <https://docs.aws.amazon.com/codedeploy/>
- Amazon Web Services. (2025). AWS Skill Builder: Cloud Practitioner Learning Plan. AWS Training and Certification. Recuperado de <https://skillbuilder.aws/>